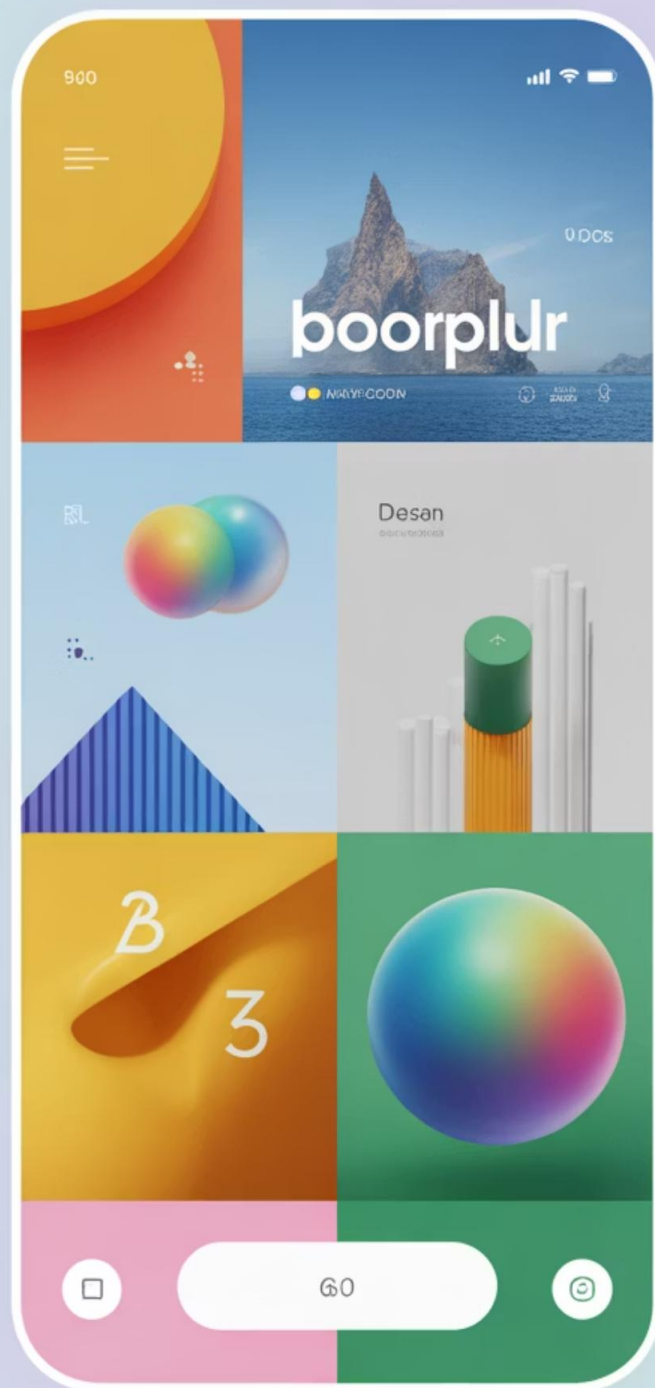


# PI IV - DESENVOLVIMENTO PARA DISPOSITIVOS MÓVEIS

Prof. Me. Wilson Lourenço



## React Native: Layout com Flexbox

- Conceitos fundamentais de Flexbox no React Native e interfaces móveis responsivas.

# Por que Flexbox no React Native?



## Layout Consistente

- Flexbox oferece layout consistente em diferentes tamanhos e orientações de tela, adaptando-se automaticamente a dispositivos variados.



## Abordagem React Native

- React Native usa Flexbox com algumas diferenças importantes em relação ao CSS web, otimizado para o ambiente mobile.



## Interfaces Responsivas

- Essencial para criar interfaces responsivas e flexíveis em apps móveis que funcionam perfeitamente em qualquer dispositivo.

# O que é Flexbox?

## Modelo de Layout Inteligente

- Flexbox é um modelo de layout poderoso para distribuir espaço e alinhar itens de forma dinâmica em um container.
- Trabalha com dois eixos fundamentais: eixo principal (main axis) e eixo cruzado (cross axis).

## Flexibilidade Total

- Facilita o design de layouts complexos mesmo com tamanhos dinâmicos e conteúdo variável.
- Permite criar interfaces que se adaptam automaticamente ao espaço disponível sem cálculos manuais.





# Entendendo os Eixos do Flexbox

## Main Axis (Eixo Principal)

- Determina a direção principal do layout: horizontal (row) ou vertical (column).
- É o eixo ao longo do qual os itens são posicionados.

## Cross Axis (Eixo Cruzado)

- Perpendicular ao eixo principal.
- Se o main axis é horizontal, o cross axis é vertical, e vice-versa.

# Flex Container e Flex Items

01

## Flex Container

- Componente pai que controla o layout dos filhos.
- Define as regras de posicionamento e distribuição de espaço.

02

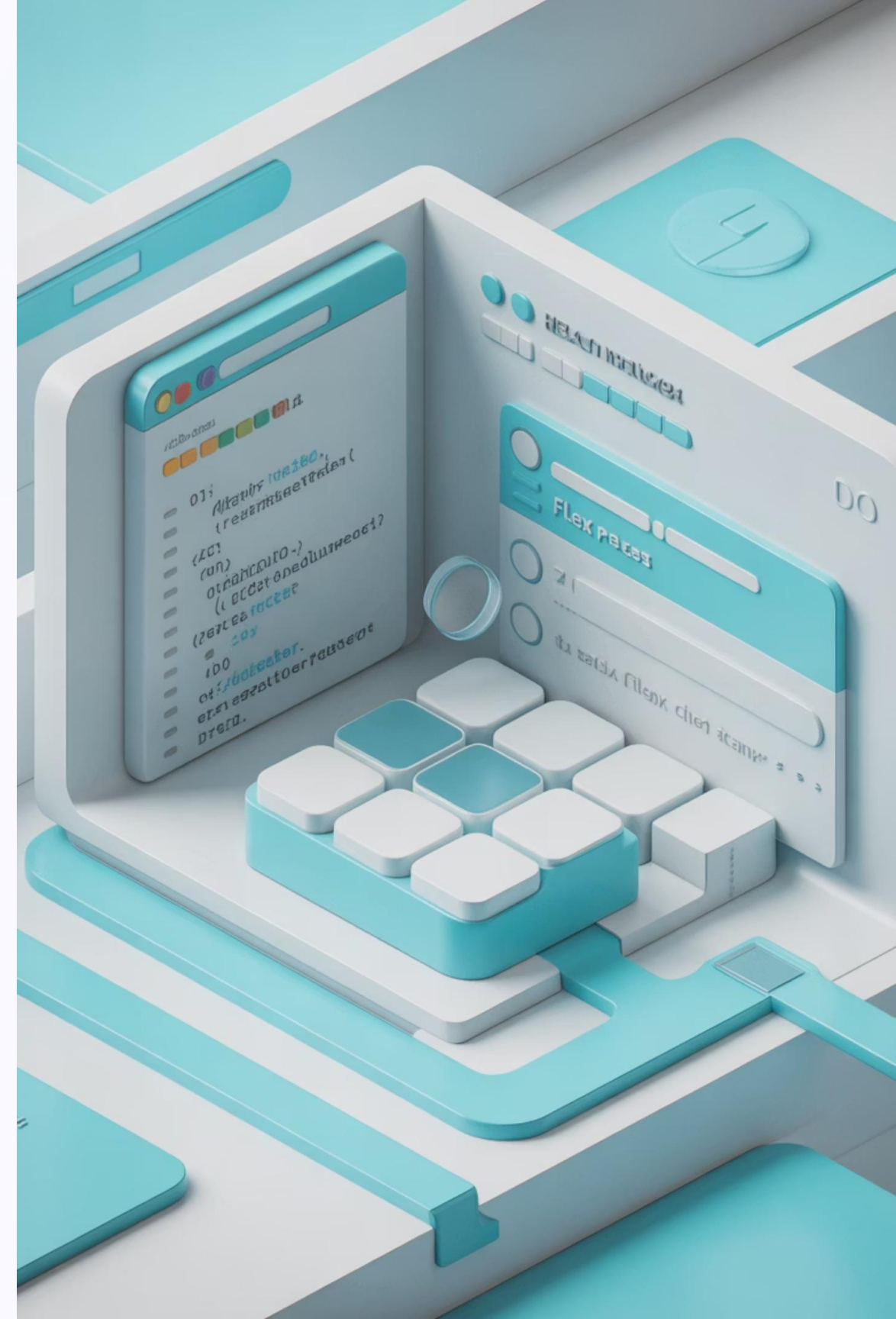
## Flex Items

- Componentes filhos que são posicionados e dimensionados dentro do container, seguindo as regras estabelecidas.

03

## Exemplo Prático

- Uma View com estilo flex:1 atua como container;
- Views internas são os itens flex que se distribuem no espaço disponível.



# Propriedade flexDirection

## Controle da Direção

- A propriedade flexDirection controla a direção do eixo principal, determinando como os itens são organizados.
- Define se seus componentes serão empilhados verticalmente ou alinhados horizontalmente.

### column (padrão)

- Itens empilhados verticalmente, de cima para baixo

### row

- Itens alinhados horizontalmente, da esquerda para direita

### column-reverse

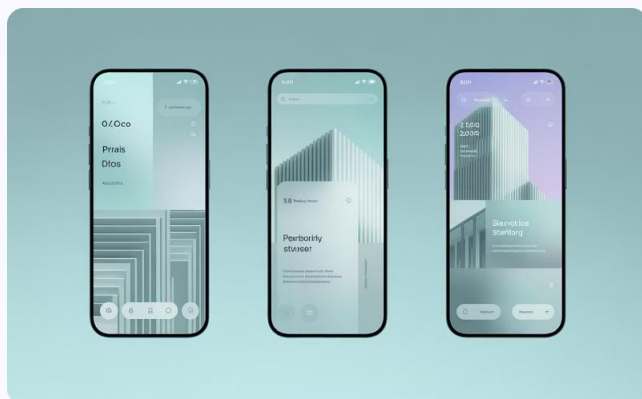
- Itens empilhados verticalmente, de baixo para cima

### row-reverse

- Itens alinhados horizontalmente, da direita para esquerda

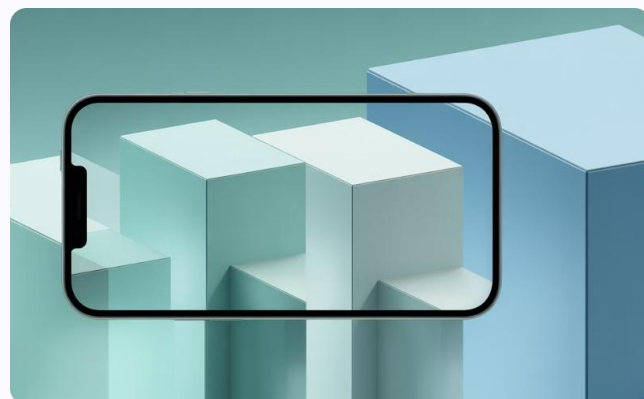
# Visualizando flexDirection

Veja como a mesma estrutura de três caixas coloridas se comporta com diferentes valores de flexDirection:



column

Alinhamento vertical  
padrão



row

Alinhamento horizontal



column-reverse

Vertical invertido



row-reverse

Horizontal invertido

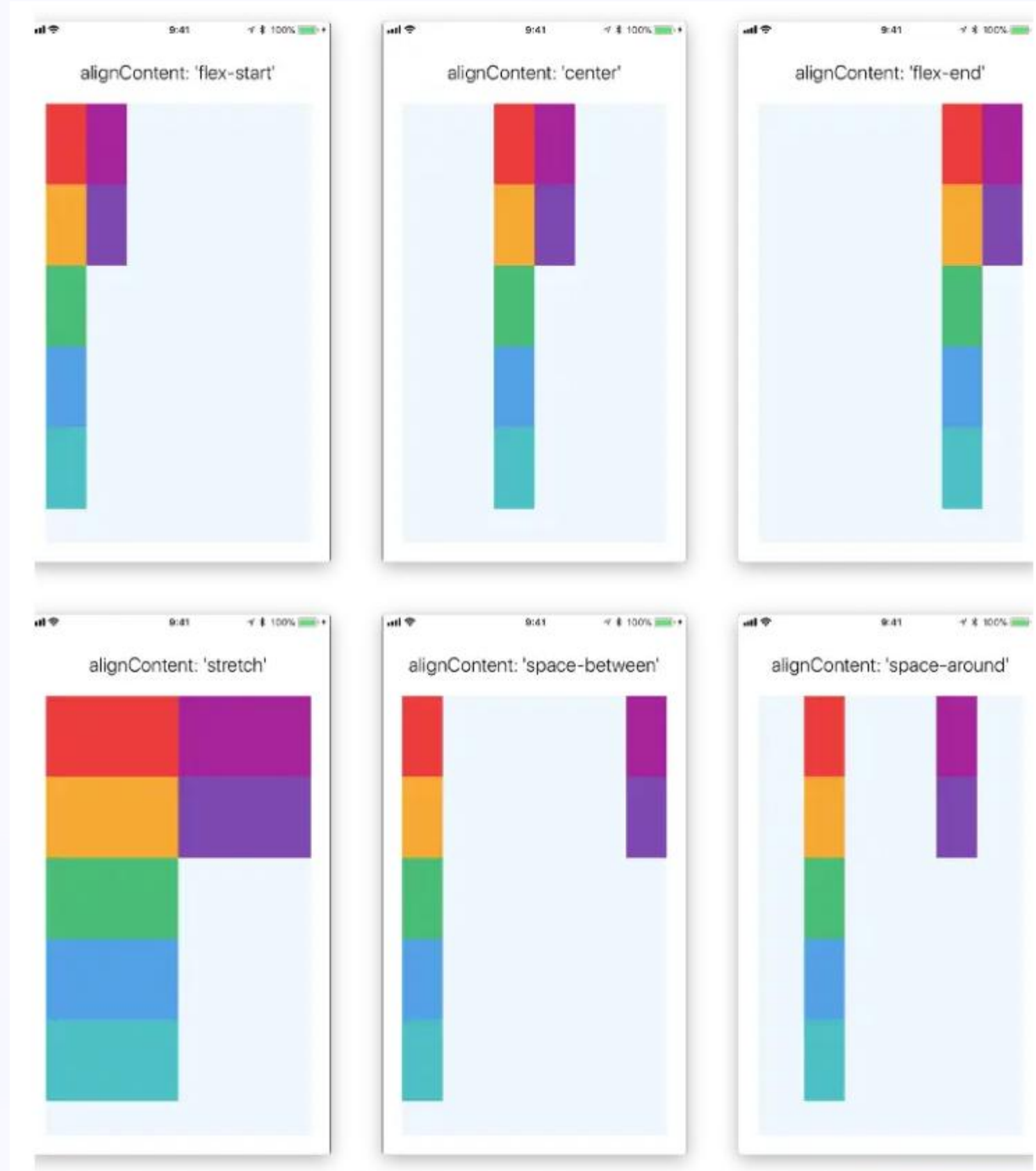


## Comparação Visual: Todas as Direções

- Observe como a mesma estrutura de componentes se transforma completamente dependendo do valor de flexDirection.
- Esta propriedade é fundamental para construir layouts que se adaptam às suas necessidades específicas.

# Propriedade alignContent

Alinha os itens ao longo do eixo principal, controlando a distribuição de espaço entre eles.



# Propriedade flex

A propriedade flex define como os itens crescem ou encolhem para preencher o espaço disponível no container.

## Valor Único

Aceita um número único que representa a proporção do espaço que o item deve ocupar em relação aos outros itens.

## Distribuição Proporcional

Itens com flex: 1, flex: 2, flex: 3 distribuem espaço na proporção 1:2:3, criando layouts balanceados e responsivos.





## Visualizando a Proporção com flex

1

Primeira Caixa

Ocupa  $\frac{1}{6}$  do espaço  
total disponível

2

Segunda Caixa

Ocupa  $\frac{2}{6}$  (ou  $\frac{1}{3}$ ) do  
espaço total

3

Terceira Caixa

Ocupa  $\frac{3}{6}$  (ou  $\frac{1}{2}$ ) do  
espaço total

# Outras propriedades importantes



## flexWrap

- Controla se os itens podem quebrar linha quando não há espaço suficiente.
- Valores: nowrap (padrão), wrap, wrap-reverse.



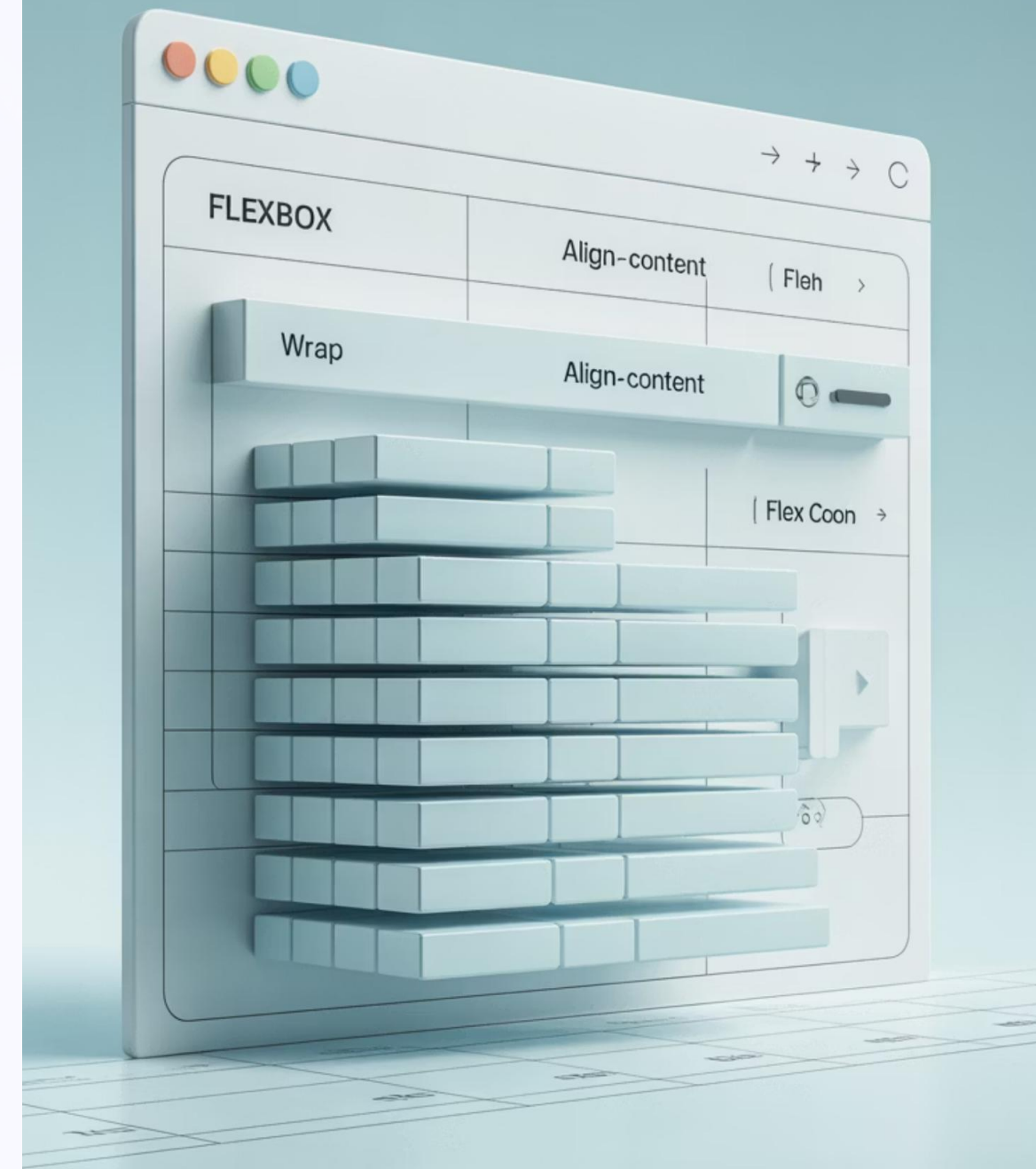
## alignContent

- Alinha linhas múltiplas no eixo cruzado quando flexWrap está ativo.
- Valores: space-between, center, stretch, flex-start, flex-end.



## alignSelf

- Sobrescreve alignItems para um item específico, permitindo alinhamento individualizado.
- Útil para destacar elementos.



# Diferenças do Flexbox no React Native vs CSS Web

Embora baseado no Flexbox do CSS, React Native possui algumas diferenças que você precisa conhecer:

1

## flexDirection padrão

- No React Native é **column** (vertical), enquanto no CSS web é **row** (horizontal).

2

## flexShrink padrão

- No React Native é **0** (itens não encolhem), enquanto no CSS web é **1** (itens encolhem).

3

## Propriedade flex

- Aceita apenas um **número único**, não a notação shorthand do CSS (flex-grow, flex-shrink, flex-basis).

4

## Layout direction

- Padrão é **LTR** (left to right), mas pode ser alterado para RTL em idiomas que exigem leitura da direita para esquerda.

## Para rodar no VS Code

Abra o terminal e digite:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy  
RemoteSigned
```

```
npx create-expo-app prjEstruturaBasica --template blank
```

```
Npm install
```

```
npx expo start -c
```

# Exemplo prático: Criando um layout simples com Flexbox

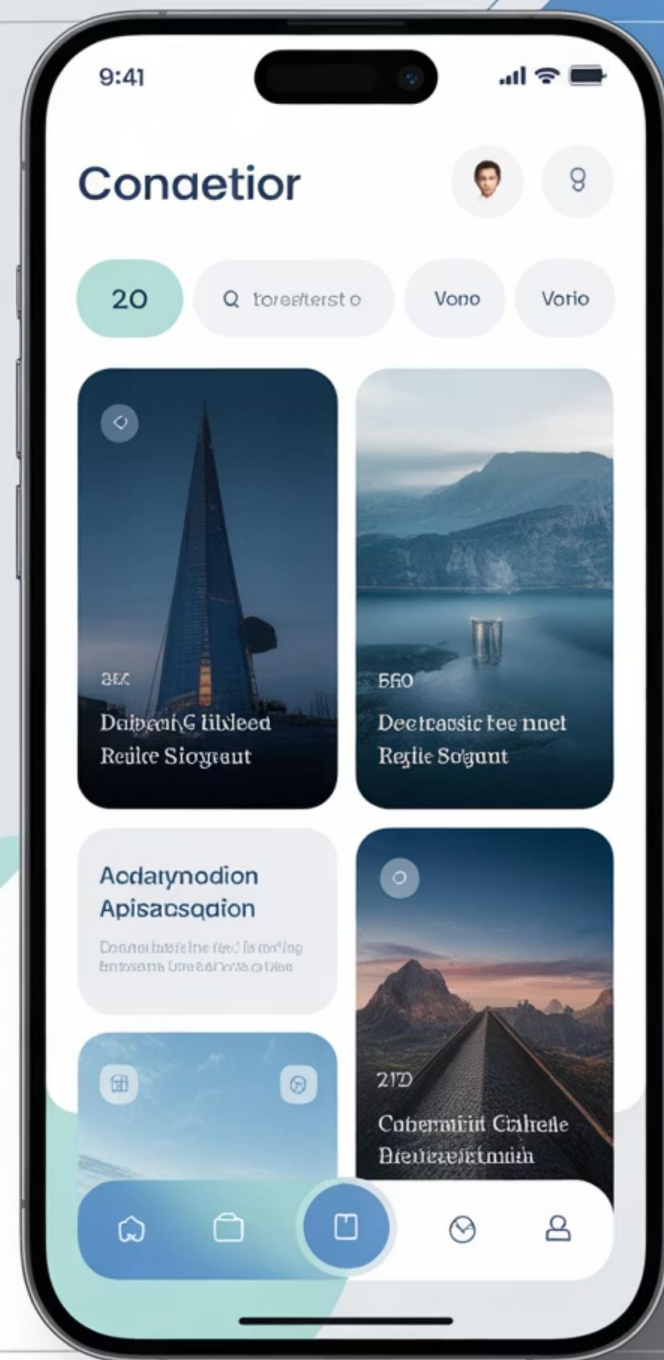
## Estrutura Básica prjEstruturaBasica

```
JS App.js M ●
prjEstruturaBasica > JS App.js > ...
You, há 2 minutos | 1 autor (You)
1 import { StatusBar } from 'expo-status-bar';
2 import { View, Text, StyleSheet } from 'react-native';
3
4 import styles from './styles';
5
6 Explain Code | Generate Tests | Generate Docstrings | Ask Sourcery
7 export default function App() {
8   return (
9     <View style={styles.container}>
10       <View style={styles.box1}>
11         <Text>Caixa 1</Text>
12       </View>
13       <View style={styles.box2}>
14         <Text>Caixa 2</Text>
15       </View>
16       <View style={styles.box3}>
17         <Text>Caixa 3</Text>
18       </View>
19     </View>
20   );
21 }
```

## Estilos Flexbox

```
JS styles.js U X
prjEstruturaBasica > JS styles.js > styles
1 import { StyleSheet } from 'react-native';
2
3 const styles = StyleSheet.create({
4   container: {
5     flex: 1,
6     flexDirection: 'column',
7     justifyContent: 'space-around',
8     alignItems: 'center',
9     backgroundColor: '#f0f0f0',
10  },
11  box1: {
12    flex: 1,
13    backgroundColor: '#FF6B6B',
14    width: '80%',
15    justifyContent: 'center',
16    alignItems: 'center',
17  },
18  box2: {
19    flex: 2,
20    backgroundColor: '#4ECDC4',
21    width: '80%',
22    justifyContent: 'center',
23    alignItems: 'center',
24  },
25  box3: {
26    flex: 1,
27    backgroundColor: '#45B7D1',
28    width: '80%',
29    justifyContent: 'center',
30    alignItems: 'center',
31  },
32 });
33
34 export default styles;
```

- Experimente este código no Expo Go ou Snacks e altere as propriedades para ver o impacto em tempo real!
- Mude flexDirection, justifyContent, alignItems e os valores de flex para explorar todas as possibilidades.



# React Native: Layout com Flexbox

Domine o modelo de layout padrão para criar interfaces responsivas e adaptáveis em React Native.

# Introdução ao Flexbox

## Modelo Padrão

- Flexbox é o sistema de layout padrão para organizar componentes no React Native.

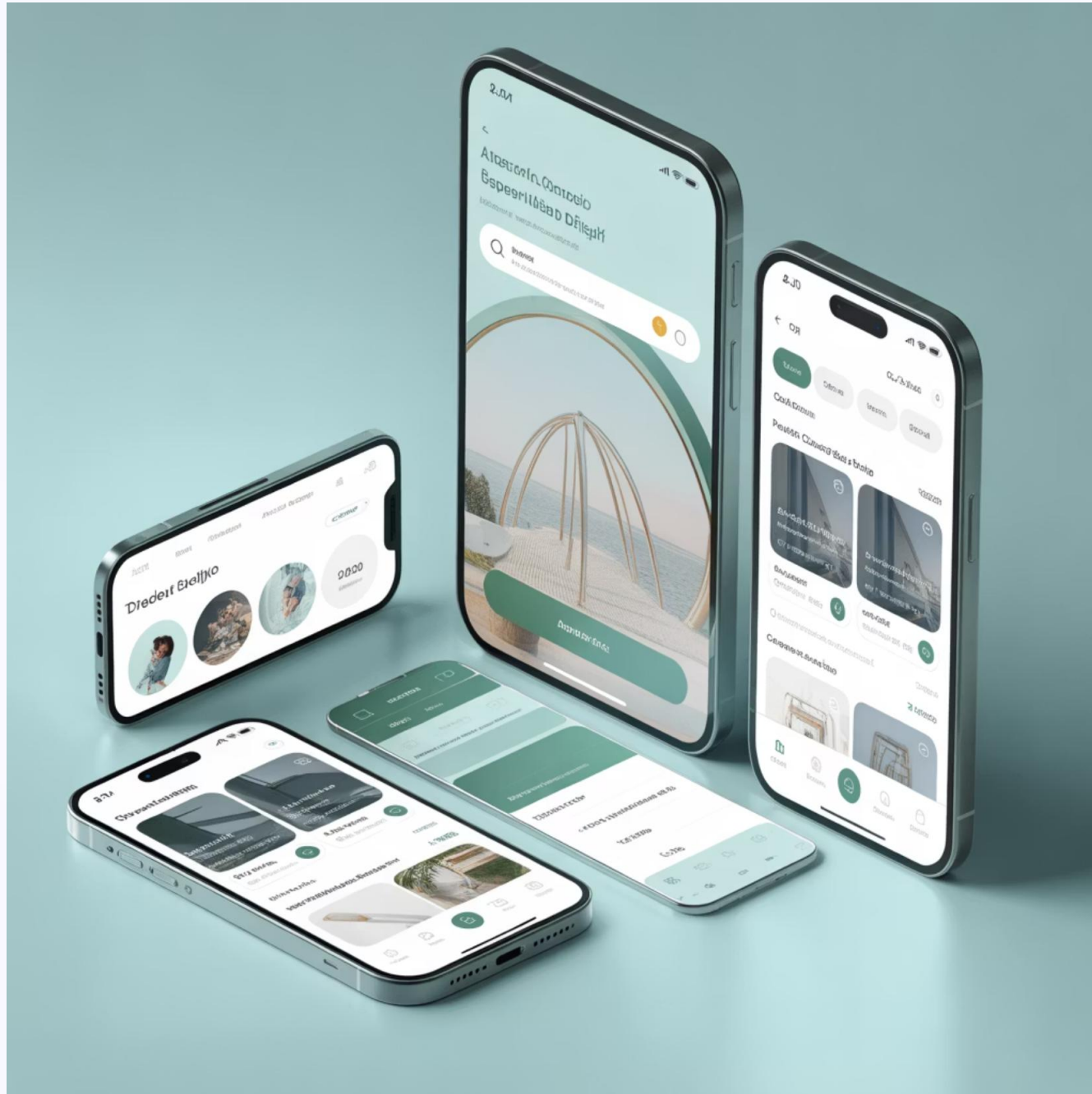
## Diferença do Web

- O padrão do flexDirection é **column** (vertical), diferente do CSS web.

## Responsividade

- Facilita layouts que se adaptam automaticamente a diferentes tamanhos de tela.

# Por que usar Flexbox?



## Layouts Flexíveis

Crie layouts adaptáveis sem precisar de medidas fixas.

## Controle Fácil

Alinhe e distribua elementos com propriedades simples e intuitivas.

## Multiplataforma

Compatível com iOS e Android usando o mesmo código.



Cross Axis

10

80

Cross Axis

# Conceitos Fundamentais

## Eixo Principal (Main Axis)

- Definido pela propriedade **flexDirection**.
- Determina a direção principal do layout.

## Eixo Cruzado (Cross Axis)

- Perpendicular ao eixo principal.
- Controla o alinhamento transversal dos elementos.

## Flex Items

- Elementos filhos dentro do container flexível que serão organizados pelo Flexbox.

Propriedade: flex

# Propriedade: flex

Define a proporção do espaço que um item ocupa no eixo principal.

1

flex: 1

Ocupa 1/6 do espaço



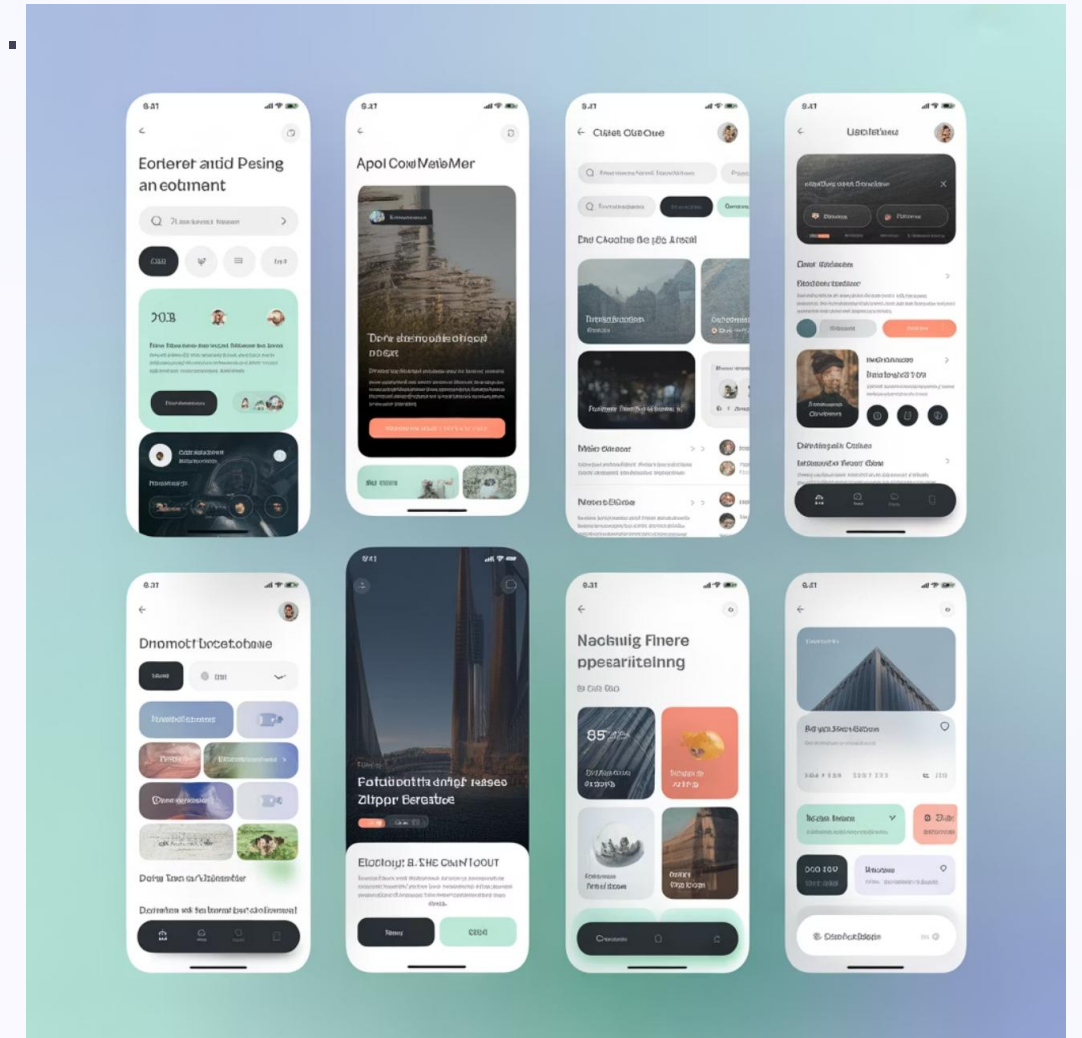
flex: 2

Ocupa 2/6 do espaço

3

flex: 3

Ocupa 3/6 do espaço



- Usado para criar layouts proporcionais sem valores fixos.

# Propriedade: flexDirection

Controla a direção do eixo principal e como os itens são organizados.



column (padrão)

Itens empilhados verticalmente

row

Itens alinhados horizontalmente



column-reverse

Vertical com ordem invertida

row-reverse

Horizontal com ordem invertida



# Propriedade: justifyContent

Alinha os itens ao longo do eixo principal

**flex-start**

Itens no início (padrão)

**flex-end**

Itens no fim

**center**

Itens centralizados

**space-between**

Espaço igual entre itens

**space-around**

Espaço ao redor dos itens

**space-evenly**

Espaço igual entre e nas bordas

# Propriedade: alignItems

Alinha os itens no eixo cruzado (perpendicular ao principal)



stretch (padrão)

Itens esticam para preencher o container



flex-start

Alinhamento no início do eixo cruzado



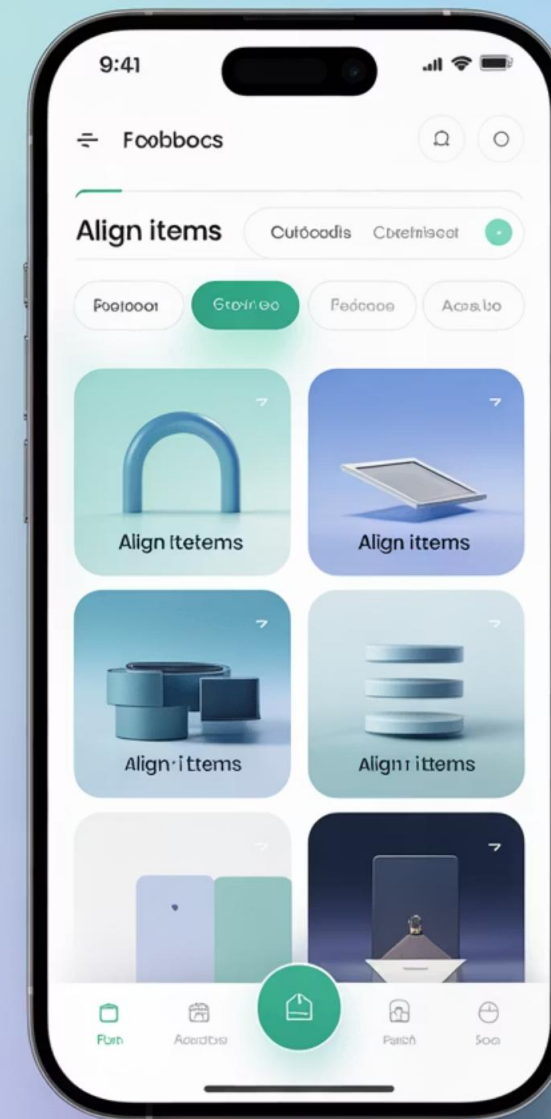
flex-end

Alinhamento no fim do eixo cruzado



center

Centraliza os itens no eixo cruzado



# Exemplo Prático

Vamos criar um layout completo usando flex, justifyContent e alignItems no Expo Go

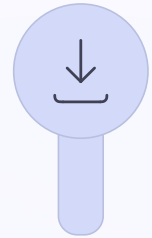
# Estrutura do Código

Exemplo prático de um layout com Flexbox no React Native: prjFlexbox

```
JS App.js M X
prjFlexbox > JS App.js > ...
You, há 2 minutos | 1 author (You)
1 import { StatusBar } from 'expo-status-bar';
2 import React from 'react';
3 import { View, Text, StyleSheet } from 'react-native';
4
5 import styles from './styles';
6
7 export default function App() {
8   return (
9     <View style={styles.container}>
10      <View style={styles.box1}>
11        <Text>Box 1</Text>
12      </View>
13      <View style={styles.box2}>
14        <Text>Box 2</Text>
15      </View>
16      <View style={styles.box3}>
17        <Text>Box 3</Text>
18      </View>
19    </View> );
}
```

```
JS styles.js U X
prjFlexbox > JS styles.js > ...
1 import { StyleSheet } from 'react-native';
2
3 const styles = StyleSheet.create({
4   container: {
5     flex: 1,
6     flexDirection: 'column',
7     justifyContent: 'center',
8     alignItems: 'center',
9     backgroundColor: '#f0f0f0',
10   },
11   box1: {
12     flex: 1,
13     width: '80%',
14     backgroundColor: '#4967E9',
15     justifyContent: 'center',
16     alignItems: 'center',
17   },
18   box2: {
19     flex: 2,
20     width: '80%',
21     backgroundColor: '#7B8FED',
22     justifyContent: 'center',
23     alignItems: 'center',
24   },
25   box3: {
26     flex: 1,
27     width: '80%',
28     backgroundColor: '#A3B4F1',
29     justifyContent: 'center',
30     alignItems: 'center',
31   },
32 });
33
34 export default styles;
```

# Testando no Expo Go



## Instale o Expo Go

Baixe o aplicativo Expo Go na App Store ou Google Play.



## Crie o Projeto

Set-ExecutionPolicy -Scope CurrentUser  
-ExecutionPolicy RemoteSigned

Use `npx create-expo-app prjFlexbox --template blank` para iniciar um novo projeto.



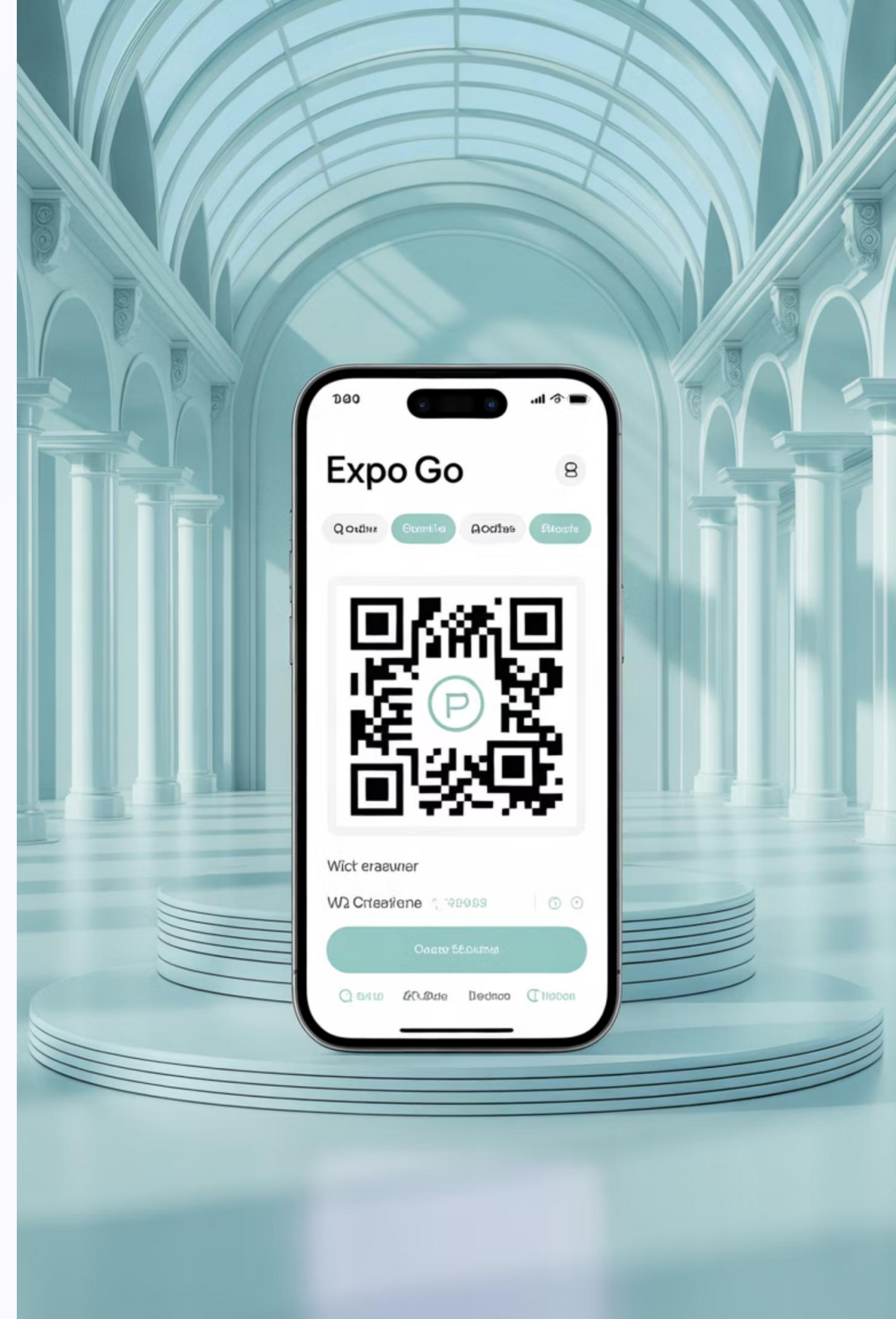
## Cole o Código

Adicione o código do exemplo no arquivo App.js.



## Execute o App

Use `npx expo start` e escaneie o QR code com o Expo Go.



# Resultado Visual

O layout criado mostra três caixas coloridas com proporções diferentes:

- Box 1 ocupa 1/4 do espaço (flex: 1)
- Box 2 ocupa 2/4 do espaço (flex: 2)
- Box 3 ocupa 1/4 do espaço (flex: 1)

Todas centralizadas horizontal e verticalmente usando **justifyContent** e **alignItems**.



# Principais Aprendizados



## flex

- Controla a proporção do espaço que cada elemento ocupa no layout.



## justifyContent

- Alinha elementos ao longo do eixo principal (vertical ou horizontal).



## alignItems

- Alinha elementos no eixo cruzado, perpendicular ao eixo principal.

- Com essas três propriedades fundamentais, você pode criar layouts responsivos e profissionais no React Native!

# React Native - Layout com Flexbox

Guia Prático e Exercício no Expo Go



# Por que Flexbox no React Native?

## Sistema Padrão

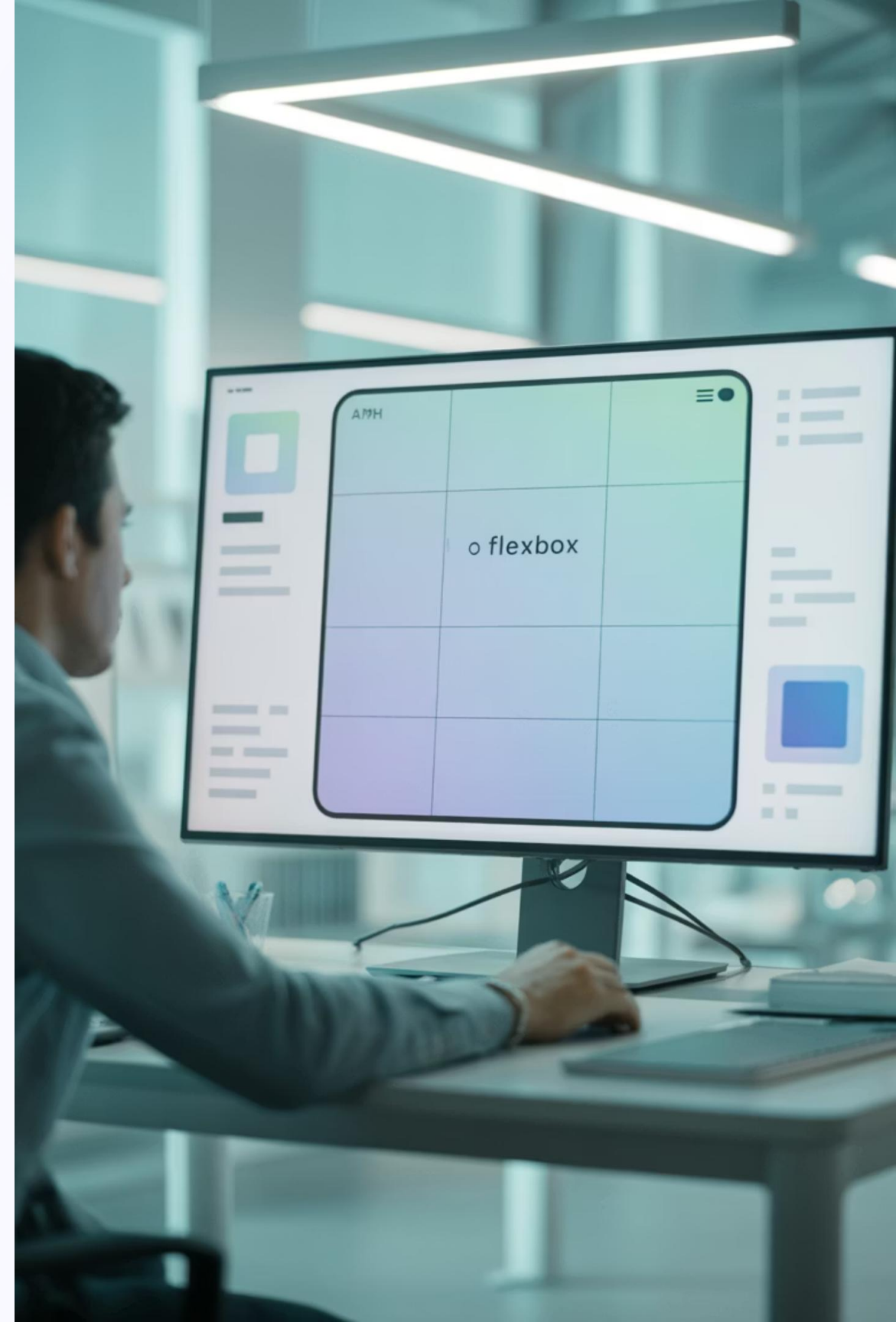
- Flexbox garante responsividade e consistência entre diferentes tamanhos de tela.

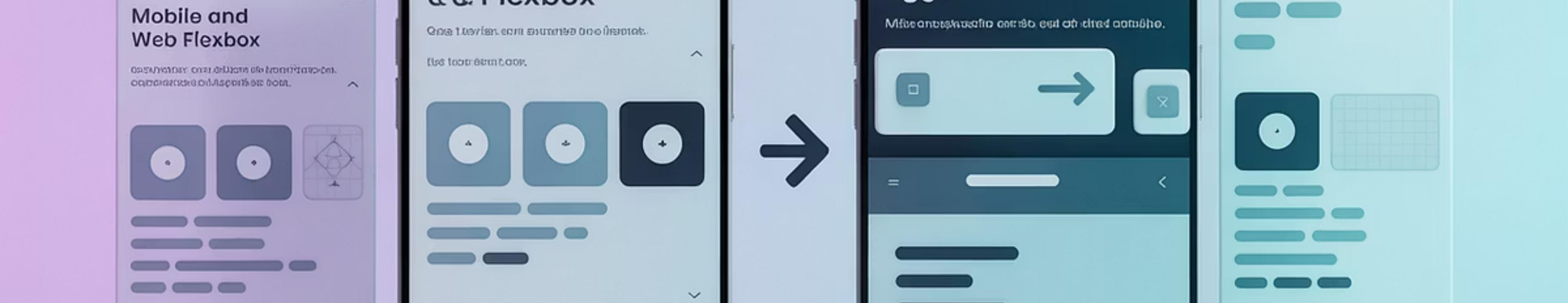
## Otimizado para Mobile

- flexDirection padrão é coluna, ideal para layouts móveis verticais.

## Desenvolvimento Ágil

- Reduz código e facilita alinhamentos complexos, acelerando o desenvolvimento.





# Flexbox: React Native vs Web

## React Native

- flexDirection padrão: **column**
- alignContent padrão: **flex-start**
- flexShrink padrão: **0**
- Flex aceita apenas número único

## Web (CSS)

- flexDirection padrão: **row**
- alignContent padrão: **stretch**
- flexShrink padrão: **1**
- Flex aceita múltiplos valores

**\*\*flexShrink** controla como um componente se ajusta quando não há espaço suficiente dentro do seu contêiner pai

Flexbox adapta seu  
layout a qualquer tela



# Propriedades Essenciais do Flexbox

1

## flexDirection

- Define o eixo principal: row, column, row-reverse, column-reverse

2

## justifyContent

- Alinha itens no eixo principal: flex-start, center, space-between, space-around, space-evenly

3

## alignItems

- Alinha itens no eixo cruzado: flex-start, center, flex-end, stretch

4

## flex

- Controla o crescimento e ocupação do espaço dos elementos

# flexDirection em Ação

## column (padrão)

- Itens empilhados verticalmente

## row

- Itens alinhados horizontalmente

## reverse

- Reversão da ordem dos itens

# justifyContent: Distribuindo Espaços



**flex-start**

Itens no início do container



**center**

Itens centralizados



**space-between**

Espaço igual entre itens



**space-around**

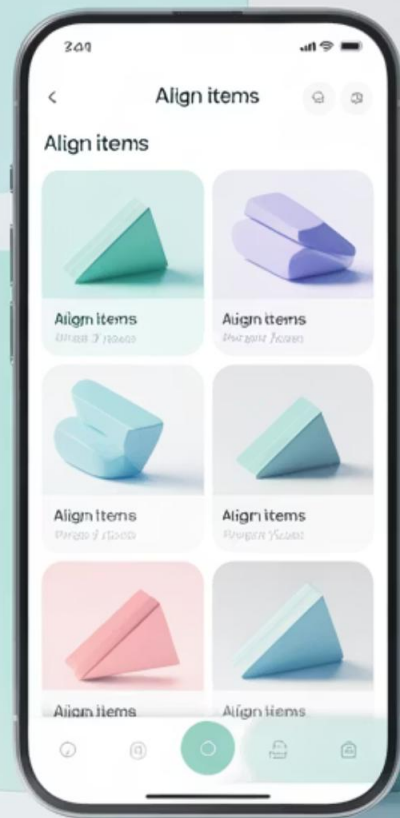
Espaço ao redor dos itens



**space-evenly**

Espaço igual entre itens e bordas

# alignItems: Alinhamento Cruzado



## flex-start

- Itens alinhados ao topo/esquerda

## flex-end

- Itens alinhados ao final

## center

- Itens centralizados no eixo cruzado

## stretch

- Itens esticam para preencher o container

# flex: Controlando Espaço Proporcional

- A propriedade flex define a fração do espaço disponível que cada elemento ocupa.

1

Primeira Caixa

- Ocupa 1/6 do espaço

2

Segunda Caixa

- Ocupa 2/6 do espaço

3

Terceira Caixa

- Ocupa 3/6 do espaço

- Flex é fundamental para layouts responsivos e adaptativos em qualquer dispositivo.



## Divisão Proporcional de Espaço

- Três caixas com flex 1, 2 e 3 dividem o espaço total em 6 partes proporcionais.

# Propriedades Avançadas e Dicas



## flexWrap

- Controla quebra de linha dos itens: nowrap ou wrap



## alignSelf

- Sobrescreve alignItems para um item específico



## Layout Direction

- LTR (padrão) e RTL para suporte a idiomas



## Responsividade

- Evite dimensões fixas para garantir adaptação

- LTR - (Left-to-Right);
- RTL (Right-to-Left).

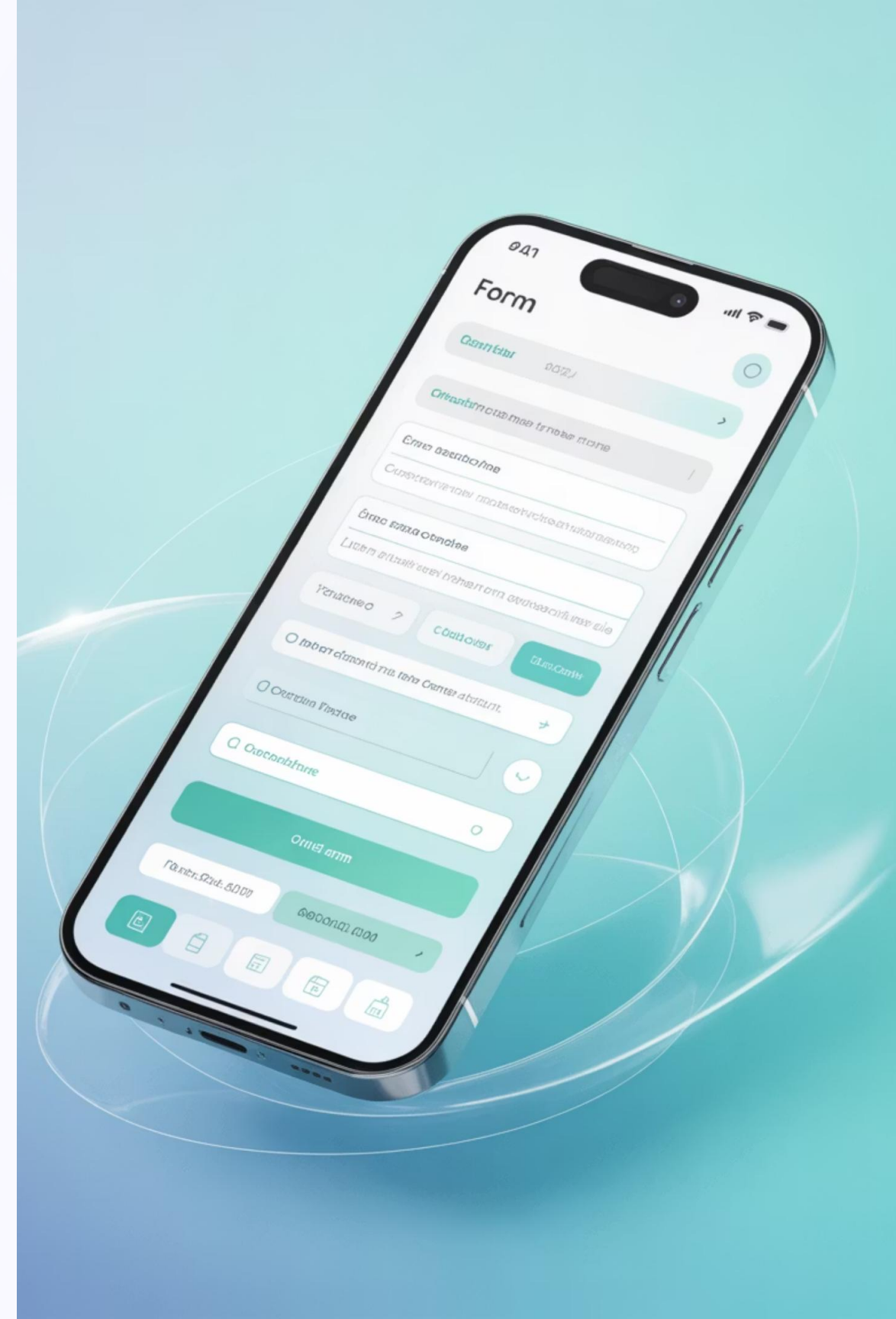


# Domine o Flexbox!

Agora você tem as ferramentas para criar layouts responsivos e elegantes no React Native. Pratique com o Expo Go e explore todas as possibilidades do Flexbox.

# React Native: Inputs e Formulários

Inputs, formulários e validação em aplicações mobile modernas



# Por que Inputs são Essenciais?



## Interação Principal

- Formulários constituem a principal forma de comunicação entre usuário e aplicativo, permitindo coleta de dados e personalização da experiência



## Componentes Nativos

- React Native oferece componentes nativos otimizados como TextInput para entrada eficiente de dados com performance nativa



## Experiência do Usuário

- Dominar inputs é fundamental para criar aplicativos responsivos, intuitivos e que proporcionem experiência fluida ao usuário

# O Componente TextInput



## Fundamentos do TextInput

O TextInput é o componente base para entrada de texto através do teclado virtual em aplicações React Native.

- Altamente configurável com diversas propriedades
- Suporta autocorreção e capitalização automática
- Oferece placeholders personalizáveis para orientação
- Eventos robustos para manipulação em tempo real

# Propriedades Essenciais

1

## autoCapitalize

- Controla a capitalização automática do texto digitado.
- Opções incluem 'none' para nenhuma capitalização, 'words' para primeira letra de cada palavra, 'sentences' para primeira letra de frases, e 'characters' para todas maiúsculas.

2

## keyboardType

- Define o tipo de teclado virtual exibido.
- Valores comuns: 'default', 'numeric', 'email-address', 'phone-pad', 'decimal-pad'.
- Essencial para otimizar a entrada conforme o contexto.

3

## placeholder

- Texto guia exibido quando o campo está vazio, orientando o usuário sobre qual informação deve ser inserida.
- Melhora significativamente a usabilidade do formulário.

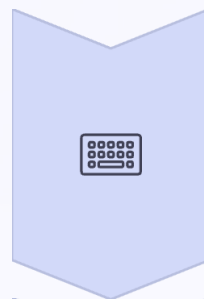
4

## secureTextEntry

- Propriedade booleana que oculta o texto digitado com asteriscos ou pontos.
- Para campos de senha e informações sensíveis, garantindo segurança visual.

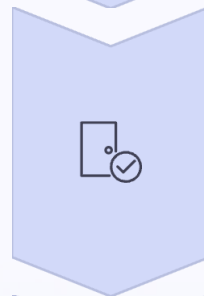


# Eventos para Manipulação de Texto



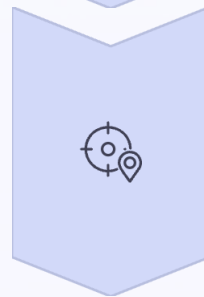
## onChangeText

- Captura o texto digitado, permitindo validação instantânea e atualização imediata do estado da aplicação conforme o usuário digita



## onSubmitEditing

- Evento disparado quando o usuário pressiona "Enter" ou "Done" no teclado virtual, ideal para processar o formulário ou mover foco para próximo campo



## onFocus / onBlur

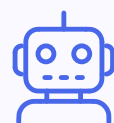
- Controle preciso de foco do campo, permitindo implementar UX aprimorada com animações, validações contextuais e feedback visual diferenciado

# Técnicas Avançadas para TextInput



## Controle Programático de Foco

- Utilize para controlar o foco, métodos `.focus()` e `.blur()` permitem navegação entre campos e melhor fluxo de preenchimento.



## Otimização para Android

- Remova bordas no Android usando a propriedade `underlineColorAndroid="transparent"`, garantindo consistência visual entre plataformas.



## Campos Multilinha

- Configure `multiline={true}` para textos longos.
- Atenção especial ao estilo das bordas para evitar problemas de renderização.



# Construindo Formulários Profissionais

# Gerenciamento de Estado em Formulários

## Abordagens para Diferentes Complexidades

- A estratégia de gerenciamento de estado depende diretamente da complexidade do formulário;
- Para formulários simples com poucos campos, o hook `useState` do React é suficiente.

# Formulários Simples

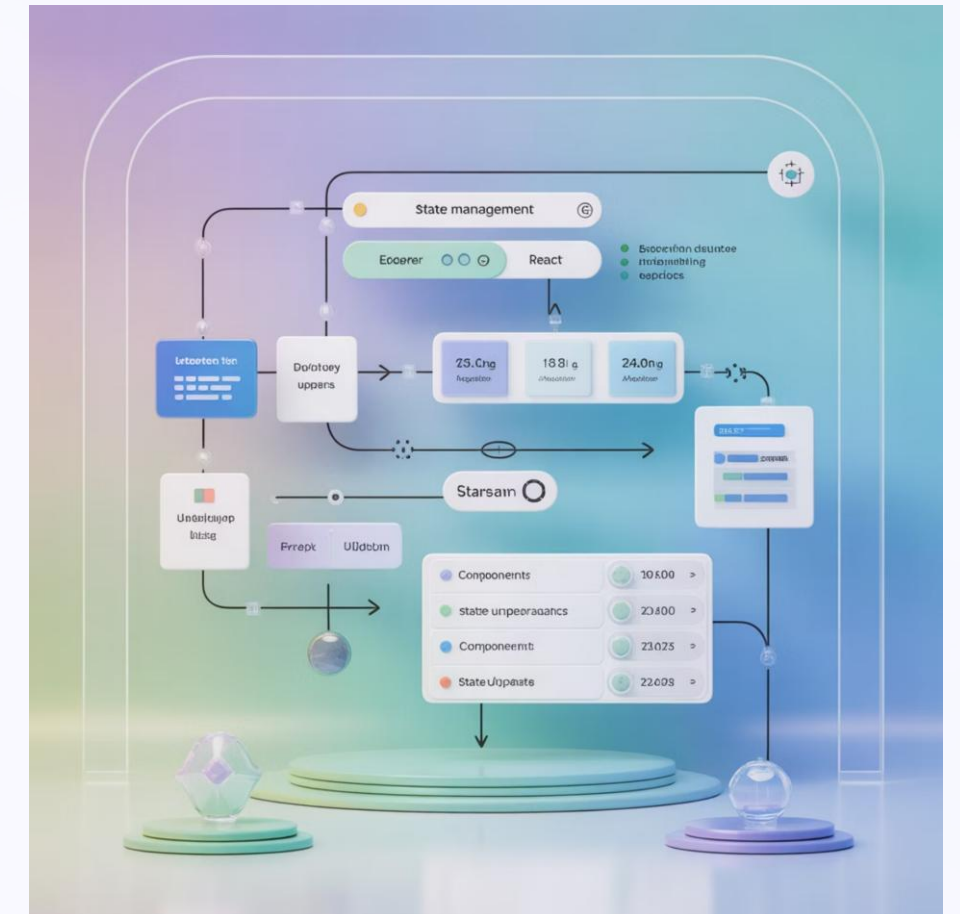
- useState para controle direto de valores individuais

# Formulários Complexos

- Bibliotecas especializadas como react-hook-form e Formik

## Validação e Feedback

- Essenciais para garantir qualidade dos dados e experiência positiva



# Exemplo Prático 1: Formulário Básico

## Implementação com TextInput e useState no Expo Go

Este exemplo demonstra um formulário simples que captura o nome do usuário, atualizando o estado em tempo real conforme a digitação.

01

### Criação do Estado

- Utilize useState para armazenar o valor do input

02

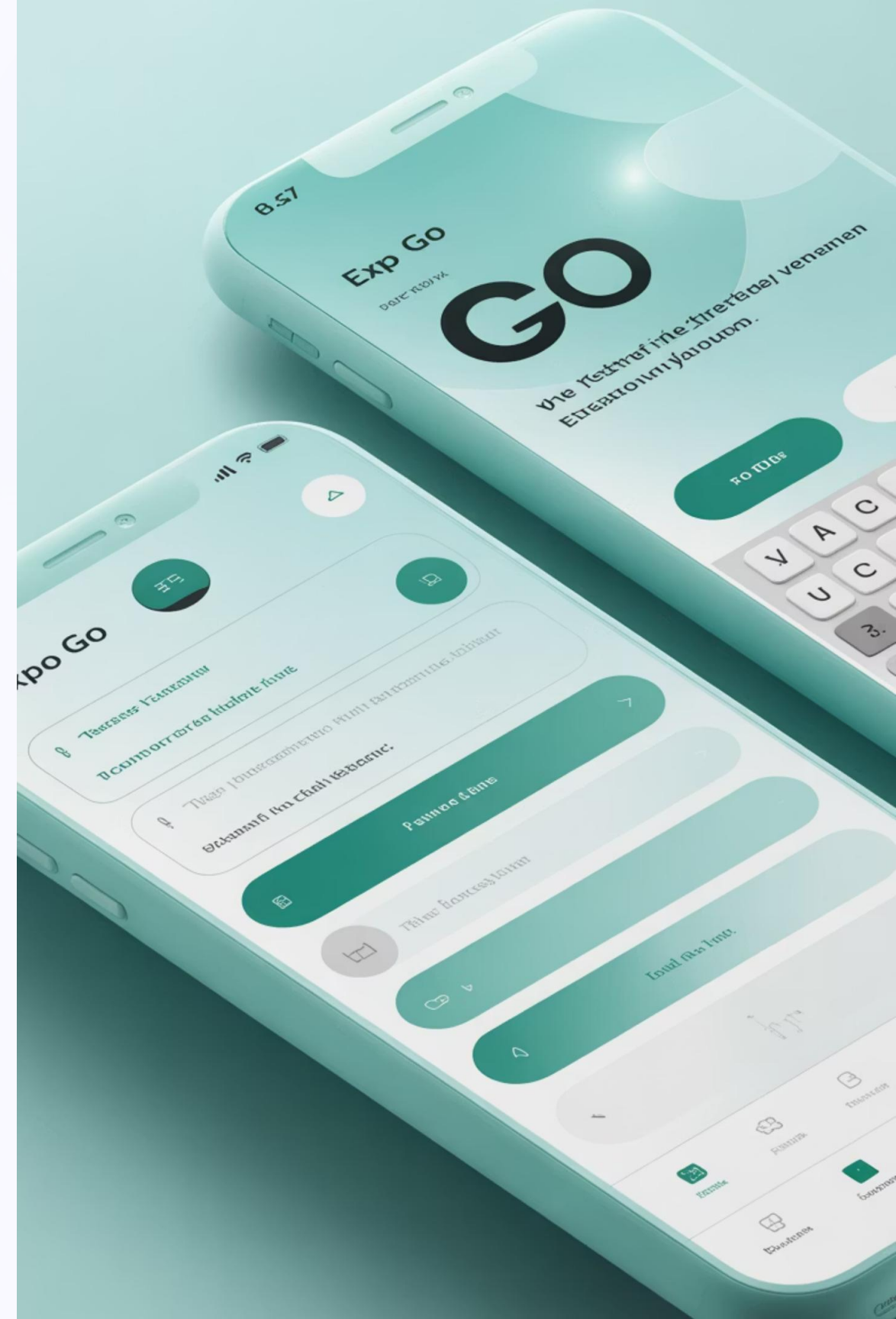
### Configuração do TextInput

- Configure placeholder e onChangeText para captura

03

### Implementação do Botão

- Botão de envio que exibe alerta com texto digitado



## Para rodar no VS Code

Abra o terminal e digite:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy  
RemoteSigned
```

```
npx create-expo-app prjFormBasico --template blank
```

```
Npm install
```

```
npx expo start -c
```

# Código do Exemplo para um formulário básico ...prjFormBasico... Expo v53.0.0

```
JS App.js M ●
prjFormBasico > JS App.js > ...
You, há 1 minuto | 1 author (You)
1 import { StatusBar } from 'expo-status-bar';
2
3 import React, { useState } from 'react'; 7.9k (gzipped: 3.1k)
4 import { View, TextInput, Button, Alert, StyleSheet } from 'react-native';
5
6 import styles from './styles';
7
8 Explain Code | Generate Tests | Generate Docstrings | Ask Sourcery
9 export default function App() {
10   const [nome, setNome] = useState('');
11   const handleSubmit = () => {
12     Alert.alert('Dados Enviados', `Nome: ${nome}`);
13   };
14   return (
15     <View style={styles.container}>
16       <TextInput style={styles.input}
17         placeholder="Digite seu nome" value={nome} onChangeText={setNome} />
18       <Button title="Enviar" onPress={handleSubmit} />
19     </View>
20   );
21 };
22
```

```
JS styles.js U X
prjFormBasico > JS styles.js > ...
1 import { StyleSheet } from 'react-native';
2
3 const styles = StyleSheet.create({
4   container: {
5     flex: 1,
6     padding: 20,
7     justifyContent: 'center'
8   },
9   input: {
10     borderWidth: 1,
11     borderColor: '#4967E9',
12     padding: 10,
13     marginBottom: 10
14   }
15 });
16
17 export default styles;
```

# Dicas para Estudo



Seja “CURIOSO”:

Procure revisar o que foi estudado.  
Pesquise as referências bibliográficas.



Seja “ANTENADO”:

Leia a próxima aula.



Seja “COLABORATIVO”:

Traga assuntos relevantes para a sala de aula.  
Participe da aula.  
Proponha discussões relevantes sobre o conteúdo.



Prof. Me. Wilson Lourenço

